# React & TypeScript

## Fundamentals for the AI-Native Builder

*A conceptual reference for non-developers shipping with AI*

Understand what AI coding agents are doing,
review their output critically,
and ship real software with confidence.

# Contents

# Introduction

## Who this is for

You want to build software. You're not a hands-on developer — maybe you've never written production code, maybe you've dabbled, maybe your job sits alongside engineering but not inside it. What's changed is that AI coding agents now do the typing for you. You're code-curious and product-minded; the bottleneck is no longer typing speed but understanding.

This document gives you the mental model: what React is, what TypeScript is, what the moving parts of a modern web application are, and what to look for when an AI agent shows you a diff. Concepts before syntax — enough syntax that you recognize what's on your screen, not enough to require memorization.

## How to use this document

Read it once linearly to build the mental model. Then keep it open as a reference while working with Claude Code, Cursor, or any AI coding agent. When the agent uses a term you don't recognize, look it up in the Glossary.

Look for three kinds of insets throughout:

- **Callouts** — the key idea or shift in thinking for a section.
- **Anti-patterns** — specific mistakes you'll encounter and want to recognize.
- **Practical tips** — direct advice from real projects.

## The vibe coding mindset

"Vibe coding" means you describe what you want; the AI writes the code. "Vibe engineering" goes further — you set the architecture, constraints, and quality bar; the AI executes within them. AI-native development means the entire workflow assumes AI agents as collaborators.

Done well, you are a director, not a typist. Done badly, you ship code you do not understand, cannot debug, and cannot evolve. The difference is fundamentals — knowing enough to spot when the AI is wrong, taking shortcuts, or solving the wrong problem. That is what this document builds.

> **Key shift**
> Your job is not to write every line. Your job is to understand every line well enough to approve it, reject it, or ask for something better. The agent is fast; you are accountable.

## What you'll be able to do after

- Read any React + TypeScript codebase and broadly follow what's happening.
- Have a substantive conversation with engineers, AI agents, or technical reviewers.

- Spot common bugs and bad patterns in AI-generated code before they ship.
- Make informed decisions about which libraries, tools, and architectures fit your project.
- Write prompts that produce maintainable code, not just code that runs.

# Part 1 — The Foundations Under React

Before React makes sense, you need a clean mental model of the layers below it. This part covers the browser, JavaScript, Node, and the toolchain.

## 1.1 How a web page actually works

A web page is three layers, each with one job:

| Layer | Job |
|---|---|
| HTML | Structure and content — headings, paragraphs, buttons, inputs |
| CSS | Presentation — colors, fonts, spacing, layout |
| JavaScript | Behavior — what happens when the user clicks, types, scrolls |

When you open a URL, your browser downloads the HTML, then any CSS and JavaScript it references. The browser parses the HTML into a tree-shaped data structure called the DOM (Document Object Model). The DOM is what you see; JavaScript can read and modify it.

> **Mental model**
> The DOM is a live tree. Every visible element is a node. React's entire job is to make updating this tree easier and faster.

## 1.2 JavaScript essentials for React

React is JavaScript. You don't need to write it, but you need to recognize these constructs:

### Variables: let and const

```
const userName = "Maya";   // cannot be reassigned
let counter = 0;            // can be reassigned
counter = counter + 1;
```

Use const by default. Use let only when the value will change.

> **⚠ Anti-pattern**
> If you see var in modern code, push back — it has scoping quirks (function-scoped instead of block-scoped) that cause subtle bugs. Modern JavaScript uses let and const exclusively.

### Functions

```
// Traditional function
function greet(name) {
  return "Hello, " + name;
}
```

```
// Arrow function (preferred in React)
const greet = (name) => "Hello, " + name;
```

Arrow functions are shorter and behave better with React's patterns. You'll see them everywhere.

### Template literals

Backtick strings let you embed values cleanly. Used constantly.

```
const name = "Maya";
const greeting = `Hello, ${name}!`;     // "Hello, Maya!"
const url = `/api/users/${id}/posts`;
```

### Objects and arrays

```
const user = { name: "Maya", age: 32, isAdmin: true };
const tags = ["react", "typescript", "learning"];

user.name        // "Maya"
tags[0]          // "react"
tags.length      // 3
```

### Destructuring

Shorthand for pulling values out of objects or arrays. Extremely common in React.

```
const user = { name: "Maya", age: 32 };
const { name, age } = user;     // name === "Maya"

const [first, second] = ["a", "b"];   // first === "a"
```

### Spread operator (...)

Copies the contents of an object or array. Used constantly to update state immutably.

```
const original = { name: "Maya", age: 32 };
const updated  = { ...original, age: 33 };  // copy + override age

const list = [1, 2, 3];
const longer = [...list, 4];               // [1, 2, 3, 4]
```

### Array methods: map, filter, find, reduce

These are everywhere in React for transforming data into UI.

```
const nums = [1, 2, 3, 4];

nums.map(n => n * 2);                    // [2, 4, 6, 8]  — transform each
nums.filter(n => n > 2);                 // [3, 4]        — keep matches
nums.find(n => n === 3);                 // 3             — first match
nums.reduce((s, n) => s + n, 0);         // 10            — combine to one
```

### Optional chaining and nullish coalescing

Two small operators that AI agents use constantly to handle missing data safely.

```
user?.address?.city          // undefined if user or address is missing
user?.greet?.()              // call only if function exists

const name = input ?? "Anonymous";   // only if input is null/undefined
```

**✓ Practical tip**

Use ?? (nullish coalescing) rather than || when defaulting to a value, because || treats 0 and "" (empty string) as falsy and replaces them. ?? only replaces null and undefined — usually what you want.

### Promises and async/await

JavaScript runs in a single thread. Operations that take time (calling an API, reading a file) return a Promise — a placeholder for a future value. The async/await syntax makes Promise-based code readable.

```
async function loadUser() {
  const response = await fetch("/api/user");
  const data = await response.json();
  return data;
}
```

**Why this matters**

Almost every AI-written component that talks to a backend will use async/await. If you see await, the line is waiting for something to complete before moving on.

**⚠ Anti-pattern**

Forgetting await is a classic AI bug. Without it, you get a Promise object instead of the value. If a variable suddenly contains "[object Promise]" or you see ".then is not a function" errors, look for a missing await.

### Modules: import and export

Modern JavaScript splits code into files (modules). One file exports things; another imports them.

```
// In utils.ts
export function formatDate(d) { return d.toISOString(); }
export const APP_NAME = "MyApp";
export default class Logger { /* ... */ }   // one default per file

// In App.tsx
import Logger, { formatDate, APP_NAME } from "./utils";
```

> ✓ **Practical tip**
> Prefer named exports over default exports. Named exports give better IDE auto-import, prevent silent renames, and refactor safely. Many teams ban default exports for this reason.

## 1.3 Node.js, npm, and the package ecosystem

JavaScript was born in the browser. Node.js lets you run JavaScript outside the browser — on your laptop, on servers, in build tools. You need Node installed to do React development, even though the React app eventually runs in a browser.

### npm: the package manager

npm (Node Package Manager) is how you install third-party code. There are over 2 million packages on the npm registry — React itself is one of them. Two files govern your dependencies:

| File | What it is |
|---|---|
| package.json | Your declared dependencies and project metadata. You read and edit this. |
| package-lock.json | Auto-generated. Pins exact versions of every dependency and sub-dependency. You commit it but don't edit it. |

Common commands you will see AI agents run:

```
npm install                    # install everything in package.json
npm install react-router-dom   # add a new dependency
npm install -D vitest          # add a dev-only dependency
npm run dev                    # start the dev server
npm run build                  # produce production files
npm run test                   # run tests
```

> **Variants**
> You may see yarn, pnpm, or bun — these are alternative package managers that do the same job. pnpm is faster and disk-efficient; bun is even faster. The concepts are identical.

### The node_modules folder

When you run npm install, every package gets downloaded into a folder called node_modules. It is enormous — often tens of thousands of files. You do not commit it to git; you regenerate it from package.json on each machine.

### Versioning and the ^ prefix

Versions look like 1.4.2 — three numbers meaning major.minor.patch.

- **Patch** (1.4.2 → 1.4.3): bug fixes, no API change.

- **Minor** (1.4.0 → 1.5.0): new features, backward compatible.
- **Major** (1.0.0 → 2.0.0): breaking changes — your code may not work after upgrade.

The ^1.4.2 in package.json means "any 1.x.x that's at least 1.4.2." The lock file pins the exact installed version.

> ⚠ **Anti-pattern**
> Mass-upgrading dependencies with npm update on a Friday afternoon. Major versions can introduce breaking changes silently. Upgrade one package at a time, review the changelog, run tests.

## 1.4 Build tools: Vite, Webpack, and bundlers

Modern web apps are written in many small files, with TypeScript and JSX (you'll meet JSX in Part 3) that browsers cannot run directly. A build tool — also called a bundler — does three jobs:

1. Transpile: convert TypeScript and JSX into plain JavaScript the browser understands.
2. Bundle: combine many files into a few optimized files so the browser doesn't make hundreds of requests.
3. Serve in dev: run a local server that watches your files and instantly refreshes the browser when you save.

Names you will encounter:

| Tool | Status |
|---|---|
| Vite | Current default for new React projects. Fast. Recommended. |
| Webpack | The veteran. Still used heavily in older projects and frameworks. |
| esbuild / SWC / Turbopack / Rollup | Underlying engines that Vite, Next.js, and others use for speed. |
| Create React App (CRA) | Older, now deprecated. Vite is the replacement. |

> **When the AI says…**
> If Claude Code suggests "npm create vite@latest", it's scaffolding a new React project using Vite. That's the modern default.

## 1.5 The browser developer tools

Every browser has built-in DevTools (right-click → Inspect, or F12). They are essential — you'll use them to debug constantly.

| Panel | What you do there | When to open it |
|---|---|---|
| Elements | Inspect the DOM, edit styles live | Layout or styling looks wrong |
| Console | See log output, type JavaScript | Anytime — errors land here |
| Network | Watch HTTP requests | API call is failing or slow |
| Sources | Set breakpoints, step through code | Deep debugging |
| Application | Inspect cookies, localStorage, etc. | Auth or storage bugs |

**✓ Practical tip**

Install the React Developer Tools browser extension. It adds a Components panel that shows your component tree, props, and state — far more useful than reading the raw DOM. Pair it with Redux DevTools or TanStack Query DevTools when those libraries are in use.

# Part 2 — TypeScript

## 2.1 Why TypeScript exists

JavaScript is dynamically typed: a variable can hold a string, then a number, then an object, with no warning. This is flexible but fragile — bugs slip through because nothing checks that you're using values the right way.

TypeScript is JavaScript with types added. You write annotations describing what each value should be; a checker verifies your code matches those declarations before it ever runs. The compiler strips the types out and produces ordinary JavaScript for the browser.

> **The deal**
> TypeScript costs you a little extra writing. It saves you a lot of debugging. For AI-assisted coding, types are doubly valuable: they give the agent guardrails about what's allowed, and they give you a precise specification of what each function expects and returns.

## 2.2 Basic types

```
let count: number = 5;
let userName: string = "Maya";
let isActive: boolean = true;
let tags: string[] = ["a", "b"];
let anything: any = "avoid this";  // disables type checking — bad
let nothing: null = null;
let notSet: undefined = undefined;
let neverHappens: never;            // can't exist (e.g. always-throw)
let dontKnowYet: unknown;           // forces a check before use
```

> **⚠ Anti-pattern**
> any is the escape hatch that ruins everything. It tells the type checker "trust me" and propagates: any value used in any expression yields any. When an AI writes any, ask for a real type. unknown is almost always the better escape hatch — it requires you to narrow before use.

## 2.3 Objects, interfaces, and type aliases

To describe the shape of an object, you use either an interface or a type alias. They are nearly interchangeable; teams pick one style and stick to it.

```
interface User {
  id: string;
  name: string;
  age: number;
  isAdmin?: boolean;   // ? means optional
}
```

```
// Equivalent with a type alias:
type User = {
  id: string;
  name: string;
  age: number;
  isAdmin?: boolean;
};

const maya: User = { id: "u1", name: "Maya", age: 32 };
```

## 2.4 Union types and literal types

A union says "this can be one of several types."

```
let id: string | number;       // either is fine
id = "abc";
id = 42;

type Status = "loading" | "success" | "error";
let s: Status = "loading";     // only those three strings allowed
```

The Status example is a literal union. It expresses an enum-like set of allowed values. You'll see this constantly for button variants, request states, theme names.

## 2.5 Discriminated unions

A more powerful pattern: union types where each variant has a shared field that tells TypeScript which variant you're in. This makes async state, redux actions, and form states type-safe.

```
type RequestState<T> =
  | { status: "idle" }
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; error: string };

function render(state: RequestState<User>) {
  if (state.status === "loading") return <Spinner />;
  if (state.status === "error")   return <Error msg={state.error} />;
  if (state.status === "success") return <Profile user={state.data} />;
  return <Idle />;
}
```

> **✓ Practical tip**
>
> Discriminated unions kill an entire class of bugs: you can no longer accidentally read data while loading, or error while success. Ask AI agents to model async or multi-state UIs this way — it's a strong upgrade over multiple boolean flags.

> ⚠ **Anti-pattern**
> Booleans for state: isLoading, hasError, isSuccess, isEmpty — four booleans give 16 combinations, most of them meaningless or contradictory. One discriminated union expresses the four valid states and nothing else.

## 2.6 Function types

```
function add(a: number, b: number): number {
  return a + b;
}

// Arrow function with type annotation:
const greet = (name: string): string => `Hello, ${name}`;

// Describing a function as a value:
type Greeter = (name: string) => string;
```

## 2.7 Generics

Generics let you write code that works with many types without losing type safety. The classic example is a list.

```
function first<T>(items: T[]): T | undefined {
  return items[0];
}

first<string>(["a", "b", "c"]);   // returns string
first<number>([1, 2, 3]);          // returns number
```

Read T as "some type — whichever the caller chose." You will see this most with React state (useState<User>) and data-fetching libraries (useQuery<User[]>).

## 2.8 Utility types

TypeScript ships with built-in type helpers that transform other types. The handful you'll meet constantly:

| Helper | What it does | Example |
|---|---|---|
| Partial<T> | All properties optional | Partial<User> for form drafts |
| Required<T> | All properties required | Strip optional from a type |
| Pick<T, K> | Subset by keys | Pick<User, "id" \| "name"> |
| Omit<T, K> | Exclude keys | Omit<User, "password"> for public profile |
| Readonly<T> | All properties read-only | Immutable view of a type |

| Helper | What it does | Example |
|---|---|---|
| Record<K, V> | Object with keys K, values V | Record<string, number> for a counter map |
| ReturnType<F> | The return type of a function | ReturnType<typeof getUser> |
| Awaited<T> | Unwrap a Promise's value | Awaited<ReturnType<typeof getUser>> |

**✓ Practical tip**

Building API client types? Define one User and derive everything else: type CreateUserInput = Omit<User, "id" | "createdAt">. Now adding a field to User automatically updates the input type, the patch type, the public profile — no drift.

## 2.9 Type inference

TypeScript does not require annotations on everything. If it can figure out the type from context, you can skip the label.

```
const count = 5;            // inferred as number
const names = ["a", "b"];   // inferred as string[]
const user = { id: 1 };     // inferred as { id: number }
```

Good TypeScript code is mostly inferred, with explicit annotations on function parameters, public function return types, and exported types. Over-annotated code is noisy; under-annotated code is unsafe.

**✓ Practical tip**

Trust inference. Annotate function parameters and exported types; let the rest infer. Hovering over a variable in VS Code shows you the inferred type — use that as your check rather than typing every annotation by hand.

## 2.10 Strict mode and tsconfig

TypeScript's behavior is configured in tsconfig.json. The single most important flag is "strict": true — it turns on a bundle of stricter checks (no implicit any, strict null checks, and more). Always start a project with strict mode on.

**⚠ Anti-pattern**

Disabling strict mode to silence errors is a debt that compounds. Each time it bails you out today, it hides a bug tomorrow. If a specific file is noisy, narrow the relaxation to that file, not the whole project.

## 2.11 What to watch for in AI-written TypeScript

- Excessive any — usually means the agent gave up on the right type. Ask for a real one.
- @ts-ignore or @ts-expect-error comments — these silence the type checker. Every one is a debt. Ask why.
- **Type assertions with** `as` — sometimes legitimate, sometimes hiding a bug. Treat `as unknown as Whatever` as a red flag.
- Mismatched API types — if the backend says id is a number and the TypeScript says string, something is wrong.
- Types that look right but aren't validated at the boundary — when data crosses from outside (an API, localStorage, a URL), validate it with Zod or similar; the TypeScript types alone are a lie if the runtime data is malformed.

# Part 3 — React: The Core Model

## 3.1 What React actually is

React is a JavaScript library for building user interfaces out of reusable pieces called components. That's it. It is not a framework — it doesn't dictate routing, data fetching, or styling. It does one thing: efficiently keep the screen in sync with your data.

## 3.2 The core idea: UI as a function of state

In a traditional approach, you would manually update the DOM: "when the user clicks this button, find the cart count element and change its text from 2 to 3." This becomes unmanageable in a complex app.

React inverts the model. You describe what the UI should look like for the current state of your data, and React figures out which DOM nodes to change.

```
UI = f(state)

// In words: the UI is a function of state.
// Change the state → React redraws the affected parts.
```

> **Why this matters**
> When reviewing AI-written code, ask: where does this piece of state live? Who reads it? Who can change it? Bad component design is almost always bad state design.

## 3.3 Components

A component is a function that returns UI. Components can use other components, like LEGO bricks.

```
function Welcome() {
  return <h1>Hello, world</h1>;
}

function App() {
  return (
    <div>
      <Welcome />
      <Welcome />
    </div>
  );
}
```

Two rules to remember:

- Component names must start with a capital letter — that's how React distinguishes them from regular HTML tags.

- A component must return exactly one root element. Wrap siblings in a fragment (an empty <>…</> tag) if needed.

> ⚠ **Anti-pattern**
> The thousand-line component. When a single component starts handling layout, data fetching, form validation, and seven dialogs, split it. A good rule: if a function exceeds one screen, ask whether it's doing more than one thing.

## 3.4 JSX: HTML inside JavaScript

That HTML-looking syntax inside the function is JSX. It's not HTML and it's not a string — it's JavaScript syntax that the build tool converts into function calls. Every JSX element becomes a React.createElement call under the hood; you almost never write that by hand.

Key differences from HTML:

| Concept | HTML | JSX |
|---|---|---|
| CSS class | class="big" | className="big" |
| Inline event | onclick="..." | onClick={handleClick} |
| Inline style | style="color: red" | style={{ color: "red" }} |
| Self-closing tags | <br>, <img> | <br />, <img /> |
| JS expressions | n/a | { anyJavaScriptExpression } |
| Labels | for="id" | htmlFor="id" |

Curly braces { } inside JSX are an escape hatch into JavaScript. Anything inside them is evaluated.

```
const name = "Maya";
return <h1>Hello, {name}</h1>;        // displays: Hello, Maya
return <p>2 + 2 = {2 + 2}</p>;         // displays: 2 + 2 = 4
```

> ⚠ **Anti-pattern**
> Using dangerouslySetInnerHTML to render strings as HTML. The name is a warning — it opens you to XSS attacks if any of that string came from user input. Render text as JSX and let React escape it for you.

## 3.5 Props: passing data into components

Props (short for properties) are how a parent component passes data to a child. They're like function arguments — read-only inside the child.

```
type GreetingProps = {
  name: string;
```

```
  excited?: boolean;
};

function Greeting({ name, excited }: GreetingProps) {
  return <h1>Hello, {name}{excited ? "!" : "."}</h1>;
}

// Used like this:
<Greeting name="Maya" excited={true} />
<Greeting name="Sam" />
```

**Pattern to recognize**

Every well-typed React component defines its props as an interface or type. When reviewing code, that props type is the component's API — read it first.

**⚠ Anti-pattern**

Prop explosions. A component taking 15 props is hard to use and a sign it's doing too much. Group related props into an object, or split the component.

## 3.6 State: data that changes over time

Props come from outside; state lives inside. State is data the component owns and can change. The useState hook is how you declare it.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

Reading the destructuring out loud: useState(0) returns a pair — the current value (count) and a setter function (setCount). Calling setCount tells React the state changed, which triggers a re-render. Naming convention: setX for the setter of x.

### State updates are asynchronous

Calling setCount(count + 1) does not immediately change count. It schedules a re-render with the new value. This trips up beginners and AI agents alike. The safe pattern when the new value depends on the old one:

```
// Risky if called twice in a row:
setCount(count + 1);
```

```
// Safe — uses the function form:
setCount(prev => prev + 1);
```

### State must be updated immutably

Never mutate state directly. Always produce a new object or array.

```
// WRONG — mutating
user.name = "New name";
setUser(user);

// RIGHT — new object
setUser({ ...user, name: "New name" });
```

> ⚠ **Anti-pattern**
> Mutating arrays or objects in state — push, splice, sort, direct field assignment — and then calling the setter. React compares by reference; the reference didn't change, so the re-render is skipped or wrong. If state "isn't updating," mutation is the first suspect.

### Derived state is not state

If a value can be computed from other state or props, don't store it in state — compute it during render.

```
// WRONG — fullName drifts out of sync when first/last change
const [first, setFirst] = useState("");
const [last,  setLast]  = useState("");
const [fullName, setFullName] = useState("");

// RIGHT — derive it
const fullName = `${first} ${last}`;
```

> ⚠ **Anti-pattern**
> Mirroring props or other state into a useState. Almost always wrong. The mirror gets stale, then you add a useEffect to keep it in sync, and now you have two bugs.

## 3.7 Hooks: the foundation of modern React

Hooks are special functions whose names start with use. They are how function components get access to React features. Two rules:

1. Call hooks only at the top level of a component (not inside loops, conditions, or nested functions).
2. Call hooks only from React components or from other hooks.

### The hooks you will see most often

| Hook | Purpose | Mental model |
|---|---|---|
| useState | Local component state | A value the component owns and can change |
| useEffect | Side effects (fetching, subscriptions, timers) | Run this code after render |
| useContext | Read from a shared context | Pull data from a higher-up provider |
| useRef | Hold a mutable value across renders | An escape hatch — usually for DOM access |
| useMemo | Cache an expensive calculation | Recompute only if inputs changed |
| useCallback | Cache a function identity | Same function reference across renders |
| useReducer | State with multiple actions | Like Redux for one component |
| useId | Generate a unique ID | For accessibility — pairing labels and inputs |

### useEffect in detail

useEffect handles anything outside React's pure render flow — fetching data, setting up subscriptions, manipulating the DOM directly, starting timers.

```
useEffect(() => {
  // This runs after the render.
  fetchUser(userId).then(setUser);

  return () => {
    // Optional cleanup — runs before next effect or on unmount.
  };
}, [userId]);  // Dependency array: re-run when userId changes.
```

The dependency array is critical:

- `[ ]` — run once, after the first render only.
- `[userId]` — run after first render and whenever userId changes.
- Omitted entirely — run after every render (almost always wrong).

> **⚠ Anti-pattern**
> Using useEffect to keep one piece of state in sync with another. If state B depends on state A, derive it — don't write an effect that watches A and writes to B. Effects for sync are a smell. Effects exist to escape React (call APIs, run timers, integrate non-React code).

**Common AI failure**
Forgetting items in the dependency array, or adding too many. ESLint with the react-hooks/exhaustive-deps rule catches most of these — make sure your project has it enabled.

## 3.8 Lists, keys, and conditional rendering

Rendering a list of data is just mapping over an array of values into an array of JSX elements.

```
const tasks = [
  { id: 1, title: "Write doc" },
  { id: 2, title: "Review PR" },
];

return (
  <ul>
    {tasks.map(task => (
      <li key={task.id}>{task.title}</li>
    ))}
  </ul>
);
```

The key prop is essential — it gives React a stable identity for each item so it can update the list efficiently when items are added, removed, or reordered.

**⚠ Anti-pattern**
Using the array index as a key. It works when items never change order or get inserted in the middle — but the moment they do, React reuses the wrong DOM nodes and you get phantom values in inputs, wrong checkboxes selected, broken animations. Always use a stable unique ID.

### Conditional rendering

```
// Ternary
{isLoggedIn ? <Dashboard /> : <LoginScreen />}

// Short-circuit (only render if truthy)
{hasError && <ErrorBanner />}

// Early return
if (isLoading) return <Spinner />;
```

**⚠ Anti-pattern**
{count && <Badge count={count} />} — if count is 0, this renders the number 0 to the page, because && returns 0 (a number, which JSX shows) when count is 0. Use {count > 0 && …} or {count ? <Badge /> : null}.

## 3.9 Handling forms and events

```
function NameForm() {
  const [name, setName] = useState("");

  function handleSubmit(e: React.FormEvent) {
    e.preventDefault();           // stop the browser default reload
    console.log("Submitted:", name);
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={name}
        onChange={e => setName(e.target.value)}
      />
      <button type="submit">Save</button>
    </form>
  );
}
```

This pattern — input.value tied to state, onChange writing back — is called a controlled component. The state is the single source of truth. For complex forms, libraries like React Hook Form save you from typing this for every field.

> **✓ Practical tip**
> Wire your <button> to type="submit" and put the handler on <form onSubmit>, not on the button's onClick. This makes "press Enter" work for free and is the accessible default.

## 3.10 The component tree and data flow

Data in React flows in one direction: down the tree, from parent to child via props. Child components communicate back up by calling callback functions that the parent provided as props.

```
function Parent() {
  const [count, setCount] = useState(0);

  return <Child count={count} onIncrement={() => setCount(c => c + 1)} />;
}

function Child({ count, onIncrement }) {
  return <button onClick={onIncrement}>{count}</button>;
}
```

**Term to know: prop drilling**
Passing a prop through many layers of components just to reach a deep child. When you see prop drilling, it's a signal to use Context (Part 4) or a state library (Part 5) — but only after the drilling becomes painful, not preemptively.

## 3.11 Accessibility (a11y) — the part AI agents forget

Accessibility is making your app work for people who can't see, can't use a mouse, or use assistive technology. It's a quality bar, increasingly a legal requirement, and the dimension AI agents skip most often.

The non-negotiables:

- Use semantic HTML: <button> for buttons, <a> for links, <form> for forms, <h1>…<h6> in order. Don't reinvent these with <div onClick>.
- Every form input has a <label> tied to it (htmlFor + matching id).
- Every image has alt text — descriptive for meaningful images, alt="" for decorative ones.
- Keyboard navigation works — every interactive element is reachable with Tab and operable with Enter or Space.
- Color is not the only signal — never use color alone to communicate state (success/error/required).

✓ **Practical tip**
After any UI change, try navigating the page using only the Tab key. If you can't reach a control, or the focus order is chaotic, that's a real bug for many users. AI agents rarely test this; you can in 30 seconds.

# Part 4 — React Patterns Beyond the Basics

## 4.1 Custom hooks

If a component has logic that you want to reuse elsewhere — fetching a user, syncing with localStorage, debouncing input — you extract it into a custom hook. By convention, the name starts with use.

```
function useLocalStorage(key: string, initial: string) {
  const [value, setValue] = useState(
    () => localStorage.getItem(key) ?? initial
  );

  useEffect(() => {
    localStorage.setItem(key, value);
  }, [key, value]);

  return [value, setValue] as const;
}

// Now any component can do:
const [theme, setTheme] = useLocalStorage("theme", "light");
```

> ✓ **Practical tip**
> Custom hooks are how mature React codebases stay clean. When asking an AI to add a feature that touches state plus side effects, asking for it as a custom hook often yields better, more reusable code than inline logic in a component.

## 4.2 Context: avoiding prop drilling

Context is React's built-in mechanism for sharing data across the tree without passing props at every level. You create a context, wrap part of the tree in its Provider, and any descendant can read it with useContext.

```
const ThemeContext = createContext<"light" | "dark">("light");

function App() {
  return (
    <ThemeContext.Provider value="dark">
      <Page />
    </ThemeContext.Provider>
  );
}

function DeepChild() {
  const theme = useContext(ThemeContext);
  return <div className={theme}>...</div>;
}
```

Context is great for things that are truly global — current user, theme, language. It's the wrong tool for things that change frequently across many components — use a state library instead (Part 5).

> ⚠ **Anti-pattern**
> Putting fast-changing state in Context. Every consumer re-renders whenever any value in the Context changes — even if they only read one field. For frequently-changing values, use a proper state library (Zustand, Jotai) instead.

## 4.3 Refs: the DOM escape hatch

Sometimes you need to interact with the actual DOM element — focus an input, measure its size, integrate a non-React library. Refs are how.

```
function NameInput() {
  const inputRef = useRef<HTMLInputElement>(null);

  useEffect(() => {
    inputRef.current?.focus();
  }, []);

  return <input ref={inputRef} />;
}
```

Refs do not trigger re-renders when changed. Use them for imperative things React doesn't manage.

> ⚠ **Anti-pattern**
> Using refs to read input values instead of state. That works once, but you lose validation, conditional UI, and the React mental model. Use state for values; use refs for imperative DOM actions (focus, scroll, measure).

## 4.4 Performance: memoization

By default, when a parent re-renders, all its children re-render — even if their props didn't change. For most components this is fine; React is fast. When it isn't, you have three tools:

| Tool | What it caches | When to use |
|---|---|---|
| React.memo | A whole component | Component with expensive render and stable props |
| useMemo | A computed value | Calculation is expensive, inputs rarely change |
| useCallback | A function identity | Passing a function to a memoized child |

> ⚠ **Anti-pattern**
> Wrapping every function in useCallback and every value in useMemo "for performance." Memoization itself has overhead — you've added complexity and may have made things slower. Profile first, optimize the specific bottleneck. AI agents often sprinkle these everywhere; push back unless there's a measured reason.

> **React Compiler**
> A newer experimental React feature that automatically memoizes for you, eliminating most manual useMemo/useCallback. As it matures, manual memoization will become rare.

## 4.5 Error boundaries

If a JavaScript error occurs in a React component, the whole UI can unmount. An error boundary is a special component that catches errors in its children and shows a fallback UI instead.

In current React, libraries like react-error-boundary provide a clean API. Wrap the top of your app — or any risky section — to keep failures contained.

> ✓ **Practical tip**
> Wrap each major page or feature in its own error boundary, not just one at the root. A crashing chart shouldn't take down the whole dashboard.

## 4.6 Composition over inheritance

React strongly prefers composition (combining small components) over class-based inheritance. A common pattern is the children prop, which lets a component accept arbitrary JSX inside it.

```
function Card({ children }: { children: React.ReactNode }) {
  return <div className="card">{children}</div>;
}

<Card>
  <h2>Title</h2>
  <p>Body</p>
</Card>
```

More advanced: compound components — a parent and matched children that work together.

```
<Tabs defaultValue="profile">
  <Tabs.List>
    <Tabs.Trigger value="profile">Profile</Tabs.Trigger>
    <Tabs.Trigger value="settings">Settings</Tabs.Trigger>
  </Tabs.List>
  <Tabs.Content value="profile">Profile content</Tabs.Content>
```

```
  <Tabs.Content value="settings">Settings content</Tabs.Content>
</Tabs>
```

This is the style used by modern UI libraries like Radix UI, shadcn/ui, and Headless UI. It reads almost like HTML.

# Part 5 — State Management

## 5.1 The state landscape

Not all state is the same. The single biggest mental model upgrade you can make in React is recognizing the different categories:

| Type of state | Examples |
|---|---|
| Local UI state | Is this modal open? What's typed in this input? Active tab. |
| Cross-component UI state | Which item is selected? Sidebar collapsed? |
| Server state (cache of remote data) | List of users from the API; the current order. |
| URL state | The current route, query parameters, filters in the URL. |
| Form state | Field values, validation errors, submission status. |
| Persistent client state | Theme preference, auth token — survives reloads. |

Each category has a best-fit tool. Mixing them — using a global store for local UI state, or local state for server data — is one of the most common ways React codebases become messy.

## 5.2 Local state: useState and useReducer

For state that lives inside one component, useState is enough. When the state has many possible transitions, useReducer scales better.

```
type Action =
  | { type: "increment" }
  | { type: "decrement" }
  | { type: "reset" };

function reducer(state: number, action: Action): number {
  switch (action.type) {
    case "increment": return state + 1;
    case "decrement": return state - 1;
    case "reset":     return 0;
  }
}

const [count, dispatch] = useReducer(reducer, 0);
dispatch({ type: "increment" });
```

✓ **Practical tip**

Reach for useReducer when you have three or more pieces of related state, or several actions that affect state in non-obvious ways. The trade is more code structure for clearer transitions.

## 5.3 Lifting state up

When two sibling components need to share state, the answer is usually to move the state to their nearest common parent and pass it down as props. This is called "lifting state up."

Lift only as high as necessary. State that lives too high causes unnecessary re-renders down the tree.

## 5.4 Global client state libraries

When state needs to be read and written by many components in different branches of the tree, you reach for a library. The main contenders today:

| Library | Style | Notes |
|---|---|---|
| Zustand | Tiny store hook | Most popular modern choice. Minimal boilerplate. |
| Redux Toolkit | Strict actions + reducers | Industry standard for large apps. Verbose but predictable. |
| Jotai | Atomic — many small atoms | Good for fine-grained updates. |
| Recoil | Atom-based (by Meta) | Less active development now. |
| MobX | Observable objects | Older, very different style. |

⚠ **Anti-pattern**

Reaching for Redux on day one of a small app. Redux Toolkit is excellent for large, complex apps, but for most projects you'll build with AI, start with useState + Context, or Zustand. You can migrate to Redux later if real complexity demands it. Reverse migrations are also common.

## 5.5 Server state: TanStack Query (React Query)

Data fetched from an API is not really client state. It's a cache of something that lives on the server. It has properties client state doesn't: it can be stale, it needs revalidation, it needs caching, it has loading and error states.

TanStack Query (formerly React Query) is the dominant library for this. Once you use it, you stop writing useEffect + useState for data fetching.

```
import { useQuery } from "@tanstack/react-query";

function UserProfile({ userId }: { userId: string }) {
  const { data, isLoading, error } = useQuery({
    queryKey: ["user", userId],
    queryFn: () => fetch(`/api/users/${userId}`).then(r => r.json()),
  });

  if (isLoading) return <Spinner />;
  if (error)     return <ErrorBanner />;
  return <h1>{data.name}</h1>;
}
```

**Big takeaway**

If an AI agent writes a useEffect to fetch data, that works — but in a production app, ask whether TanStack Query (or SWR, or your framework's data loader) would be cleaner. The cache, retries, deduplication, and refetching come for free.

**⚠ Anti-pattern**

Storing server data in Redux or Zustand. You end up reinventing caching, invalidation, refetching, and loading states — and doing it worse than TanStack Query. Keep client state and server state in separate tools.

## 5.6 URL state

Anything that should survive a page refresh or be shareable as a link — filters, search terms, current tab — belongs in the URL, not in component state. React Router and Next.js give you hooks (useSearchParams, etc.) to read and write URL parameters cleanly.

**✓ Practical tip**

If a user would expect to right-click → "Copy link" and share that exact view, the relevant state must be in the URL. The URL bar is the world's oldest and most robust state store.

## 5.7 Form state

Forms accumulate state quickly: field values, validation errors, touched fields, submission state. Two libraries dominate:

- React Hook Form — performant, uses uncontrolled inputs under the hood, great DX.
- Formik — older, controlled inputs, more boilerplate.

For schema-based validation, both pair well with Zod or Yup.

✓ **Practical tip**

Define the form schema once in Zod, infer the TypeScript type from it, and use it both client-side (for validation) and server-side (for input validation). One source of truth for the shape of your data — a massive win in any AI-driven project.

# Part 6 — Talking to APIs

## 6.1 The browser's fetch API

fetch is built into every modern browser. It returns a Promise that resolves to a Response object. You usually then call .json() to parse the body.

```
async function getUser(id: string): Promise<User> {
  const response = await fetch(`/api/users/${id}`);
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  return response.json();
}
```

> ⚠ **Anti-pattern**
> fetch does not reject on HTTP error statuses like 404 or 500 — only on network failure. Always check response.ok or the status code yourself. This is a classic AI-written bug — the agent assumes fetch behaves like axios.

## 6.2 Axios and other clients

axios is a popular alternative to fetch with friendlier defaults: automatic JSON parsing, automatic rejection on error statuses, easier request cancellation, request/response interceptors. Functionally equivalent to fetch for most apps.

## 6.3 REST vs GraphQL vs tRPC

| Style | Shape | When you see it |
|---|---|---|
| REST | Many URL endpoints; each returns a fixed shape | Most APIs today; default for new public APIs |
| GraphQL | Single endpoint; the client requests the exact shape | Larger apps with many clients (web + mobile); Apollo / urql / Relay |
| tRPC | TypeScript end-to-end; no schema file — functions are the API | Full-stack TypeScript projects; only when both ends are TS |
| gRPC | Binary protocol, Protobuf schemas | Backend-to-backend; rarely browser-to-server |
| WebSockets | Persistent connection, bidirectional | Real-time: chat, presence, live dashboards |
| Server-Sent Events (SSE) | One-way streaming from server | Live notifications, AI chat streaming |

**✓ Practical tip**

If your team writes both the backend and the frontend in TypeScript, tRPC gives you end-to-end type safety with effectively zero boilerplate — the frontend gets full autocomplete on backend functions, and refactors propagate. It's a strong default for full-stack TypeScript projects.

## 6.4 Loading, error, and success states

Every async operation has at least three visual states: loading, success, error. Many also have empty (success but no data) and stale. A good UI handles all of them explicitly.

```
if (isLoading) return <Spinner />;
if (error)     return <ErrorMessage error={error} />;
if (!data || data.length === 0) return <EmptyState />;
return <List items={data} />;
```

**Review checklist**

When the AI writes a data-fetching component, scan for: loading state, error state, empty state, and request cancellation on unmount. Missing any of these is a common bug.

**⚠ Anti-pattern**

Silent catch blocks: try { … } catch (e) {} that swallow the error and continue. Either log, surface to the user, report to an error tracker, or re-throw. "It at least doesn't crash" is not error handling.

## 6.5 Mutations and optimistic updates

Fetching data is one direction; changing data on the server is the other. Mutation libraries (mostly TanStack Query) handle this with the same elegance, including:

- Optimistic updates — instantly show the new state in the UI, then roll back if the server rejects.
- Cache invalidation — automatically refetch related queries when something changes.
- Retries and error handling baked in.

## 6.6 Runtime validation at the boundary

TypeScript types are erased before the code runs — they're a compile-time check, not a runtime guarantee. When data crosses from outside your app (an API response, localStorage, a URL parameter), the TypeScript type is a wish, not a fact.

Validate at the boundary with a runtime schema library — Zod is the leader.

```
import { z } from "zod";
```

```
const UserSchema = z.object({
  id: z.string(),
  name: z.string(),
  age: z.number().int().min(0),
});

type User = z.infer<typeof UserSchema>;   // TS type derived automatically

async function getUser(id: string): Promise<User> {
  const response = await fetch(`/api/users/${id}`);
  const json = await response.json();
  return UserSchema.parse(json);          // throws if shape is wrong
}
```

**✓ Practical tip**

Define schemas once, use them everywhere — API validation, form validation, the inferred TypeScript types. One canonical source of truth for what your data looks like. Particularly valuable when AI is writing code in multiple files that all touch the same data.

# Part 7 — Routing

## 7.1 Single-page apps and client-side routing

A traditional website navigates by loading a new HTML page from the server for each URL. A single-page application (SPA) loads once, then JavaScript intercepts navigation and swaps the rendered components — much faster, no full reloads. Routing is the part that decides which component to render for which URL.

## 7.2 React Router

React Router is the long-standing standard. The current API (v6+) is declarative — you describe your routes as JSX or as a data structure.

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        <Route path="/users/:id" element={<UserPage />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Key concepts: Link replaces <a> (it intercepts the click), useParams reads URL parameters like :id, useNavigate triggers programmatic navigation.

> ⚠ **Anti-pattern**
> Using <a href="/page"> for in-app navigation. That causes a full page reload, breaks the SPA, and loses scroll position and any in-memory state. Always use <Link to="/page"> for routes inside your app.

## 7.3 Framework routing: Next.js, Remix, TanStack Router

Full frameworks like Next.js and Remix come with their own routing built around the file system — folders and files in a special directory map to URLs. This is increasingly the default for serious React apps because it integrates routing with server-side rendering and data loading.

**Framework signal**
If a project has an app/ or pages/ folder at the root, it's likely Next.js. If routes are defined in JSX in a single App.tsx, it's React Router.

## 7.4 Protected routes and redirects

Routes that require authentication need a guard — a wrapper that checks for a session and redirects to login if absent. Patterns vary by router; the concept is the same:

```
function RequireAuth({ children }: { children: React.ReactNode }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" replace />;
  return children;
}

<Route path="/dashboard" element={
  <RequireAuth><Dashboard /></RequireAuth>
} />
```

**⚠ Anti-pattern**
Trusting client-side route guards as actual security. They're a UX nicety — they decide what to show. Real authorization must happen on the server: anyone can disable the JavaScript guard and call your API directly.

# Part 8 — Testing

## 8.1 Why testing matters more in the AI era

When an AI writes code, you can review it line-by-line, but tests give you something better: a programmatic specification of how the code should behave. If the AI changes the implementation, the tests catch regressions. If you ask the AI to refactor, the tests confirm nothing broke. Tests are the safety net under vibe coding.

## 8.2 The testing pyramid

| Level | What it tests | Tools |
|---|---|---|
| Unit | A single function or component in isolation | Vitest, Jest |
| Integration | Several pieces working together | Vitest + React Testing Library |
| End-to-end (E2E) | Full user flows in a real browser | Playwright, Cypress |

You want many unit tests (fast, cheap), fewer integration tests, and a small number of E2E tests for critical flows.

> ✓ **Practical tip**
> Identify your three or four most critical user flows — sign up, checkout, the core action your app exists for — and cover them with E2E tests. Below that, lean on integration tests for components, and unit tests for non-trivial logic. Don't aim for 100% coverage; aim for confidence in the paths that matter.

## 8.3 Vitest / Jest: the test runners

Both are test runners — they find your test files, run them, report results. Vitest is faster and integrates better with Vite; Jest is older and entrenched. Syntax is nearly identical.

```
import { describe, it, expect } from "vitest";

describe("formatDate", () => {
  it("formats ISO dates", () => {
    expect(formatDate("2024-01-15")).toBe("Jan 15, 2024");
  });
});
```

## 8.4 React Testing Library

React Testing Library (RTL) is the dominant way to test components. Its philosophy: test the component the way a user would experience it — by what's on screen, not by internal implementation.

```
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";

it("increments the counter on click", async () => {
  render(<Counter />);
  const button = screen.getByRole("button", { name: /clicked 0 times/i });
  await userEvent.click(button);
  expect(screen.getByText(/clicked 1 times/i)).toBeInTheDocument();
});
```

**⚠ Anti-pattern**

Tests that query by internal implementation details — class names, test IDs sprinkled everywhere, container.querySelector chains. They break the moment you refactor without behavior changing. Good tests survive refactors; bad tests punish them.

## 8.5 End-to-end: Playwright

E2E tests launch a real browser, navigate your app, and assert on what's visible. Playwright is the modern leader (Cypress is the alternative). Slow to run but priceless for critical flows like sign-up and checkout.

## 8.6 Mocking

In tests, you usually don't want to call real APIs. Mocking replaces a real function or module with a fake. Mock Service Worker (MSW) is the gold standard — it intercepts fetch calls at the network layer so your code under test doesn't know it's being mocked.

**✓ Practical tip**

When you use MSW, the same mock definitions can power your tests, your Storybook stories, and even your dev environment when the backend is down. One mock layer, used everywhere — a high-leverage investment.

## 8.7 What to ask for when AI writes tests

- Cover the happy path, at least one error path, and the empty/edge case.
- Query by accessible roles and labels — not by classes or test IDs.
- Don't test trivial getters or implementation details — test behavior.
- Each test name reads as a user-visible statement: "shows error when login fails," not "calls setError."

# Part 9 — Production Concerns

## 9.1 Folder structure

There is no one true structure. Two common patterns:

```
// Pattern A — by type
src/
  components/
  hooks/
  pages/
  utils/

// Pattern B — by feature
src/
  features/
    auth/
      AuthForm.tsx
      useAuth.ts
      auth.api.ts
    cart/
      Cart.tsx
      cart.api.ts
  shared/
```

Pattern B (by feature) scales better. As soon as you have more than a handful of features, grouping by feature beats grouping by type.

> ✓ **Practical tip**
> Decide your folder structure on day one and put it in CLAUDE.md or AGENTS.md. AI agents tend to create new components wherever they feel like; a documented convention keeps the codebase consistent.

## 9.2 Linting and formatting

| Tool | Job |
|---|---|
| ESLint | Catches bugs and style violations in code |
| Prettier | Auto-formats code consistently — no debates about spacing |
| Biome / Oxc | Newer, faster alternatives that combine both |

A well-configured project runs these on save and rejects pull requests that fail. AI agents respect the configuration if it's present, so investing in good lint rules pays off.

## 9.3 Environment variables

Values that change between environments (dev, staging, production) — API URLs, feature flags, public keys — live in .env files.

```
# .env.local
VITE_API_URL=https://api.example.com
VITE_ANALYTICS_KEY=pk_live_xxx

// Used in code:
const apiUrl = import.meta.env.VITE_API_URL;     // Vite
const apiUrl = process.env.NEXT_PUBLIC_API_URL;  // Next.js
```

**⚠ Anti-pattern**

Committing .env files to git. They contain environment-specific configuration and often slip in secrets. Add .env* to .gitignore (except .env.example) and use your hosting provider's secret manager for production values.

**Security**

Anything in a frontend bundle is public. Never put true secrets — server API keys, database passwords — in frontend env vars. Vite requires the VITE_ prefix and Next.js requires NEXT_PUBLIC_ to flag this explicitly. Anything without that prefix is server-side only.

## 9.4 Building for production

npm run build produces an optimized bundle: minified, tree-shaken (unused code removed), and split into chunks for efficient loading. The output goes to a dist/ or build/ folder, ready to upload to a static host.

## 9.5 Code splitting and lazy loading

Don't ship every screen's code to every user. Split your bundle so each route only loads what it needs.

```
import { lazy, Suspense } from "react";

const Settings = lazy(() => import("./Settings"));

<Suspense fallback={<Spinner />}>
  <Settings />
</Suspense>
```

**✓ Practical tip**

Inspect your built bundle with a tool like vite-bundle-visualizer or webpack-bundle-analyzer. A surprisingly large chunk is almost always one accidentally imported library (a date library, an icon library, a chart library) you can replace or lazy-load.

## 9.6 Deployment

A pure React SPA is just HTML, CSS, and JS files. Anywhere that serves static files will work: Vercel, Netlify, Cloudflare Pages, AWS S3 + CloudFront, GitHub Pages. For server-rendered apps (Next.js, Remix), you need a host that can run Node.js or a framework-specific runtime — Vercel and Netlify handle this out of the box.

## 9.7 SSR, SSG, and frameworks

As apps grow, you start caring about things like SEO, initial load speed, and dynamic content from the server. The terms to know:

| Term | Acronym | What it means |
|---|---|---|
| Client-side rendering | CSR | Browser downloads JS, JS renders the page. Default for plain React. |
| Server-side rendering | SSR | Server renders the HTML for each request, ships it to the browser. |
| Static site generation | SSG | HTML is generated at build time and served from a CDN. |
| Incremental static regeneration | ISR | SSG that revalidates pages in the background after deployment. |
| React Server Components | RSC | Newer model: some components render only on the server, never ship JS to the client. |

Plain React handles only CSR. Next.js, Remix, and TanStack Start handle the rest. If your project's needs go beyond CSR — SEO, fast first load, dynamic data integrated with rendering — that's the signal to consider a framework.

## 9.8 Monitoring and error tracking

Production errors will happen. You need to know when. The names to recognize:

- Sentry — the leader for frontend error tracking. Captures stack traces, breadcrumbs, user context.
- LogRocket / FullStory — session replay. See what the user did when the error happened.
- PostHog, Mixpanel, Amplitude — product analytics, often pulling double duty for error tracking.

> ✓ **Practical tip**
> Wire up an error tracker before launch, not after the first incident. The 30 minutes of setup pays back the first time a user hits a bug you didn't know existed.

## 9.9 Performance metrics that matter

Browsers and Google measure user-perceived performance with Core Web Vitals:

| Metric | What it measures | Good |
|---|---|---|
| LCP | Largest Contentful Paint — when the main content appears | < 2.5 s |
| INP | Interaction to Next Paint — responsiveness to clicks/taps | < 200 ms |
| CLS | Cumulative Layout Shift — visual stability | < 0.1 |

Chrome's Lighthouse tool measures these on demand. PageSpeed Insights does it from real users.

# Part 10 — The Full Stack Landscape

Up to this point we've focused on the frontend — what runs in the browser. Real applications have at least one more layer (and usually several): the backend, the database, authentication, payment processing, file storage, email, analytics, deployment infrastructure. This part shows how React + TypeScript fits into the larger picture so you can navigate any real project description.

## 10.1 The split: frontend vs backend

The cleanest mental model is two halves separated by a network boundary:

| Side | Where it runs | What it does |
|---|---|---|
| Frontend | User's browser or mobile device | Render the UI, handle interactions, call the backend |
| Backend | A server you control (or a serverless function) | Business logic, database access, authentication, sensitive operations |
| Database | Usually managed by a cloud provider | Persistent storage of your data |

They talk to each other over HTTP (or WebSocket for live updates). Everything sensitive — auth secrets, payment processing, database credentials, business rules — lives on the backend. The frontend is untrusted; anything you don't enforce on the backend, you don't enforce.

> **The security rule**
> Treat the frontend as a UI for your API, not as a guard. A skilled user can modify any frontend code, intercept any request, and bypass any check you put there. Auth and authorization belong on the server, every time.

## 10.2 A typical request: what actually happens

When a user clicks "Save Profile" in your React app, the chain of events looks roughly like this:

1. React calls a mutation function in the frontend.
2. The function sends an HTTP POST to https://api.yourapp.com/users/123 with the new data and an auth token (usually in the Authorization header).
3. Request travels across the internet to your backend server (or serverless function).
4. The server verifies the auth token, checks the user is allowed to modify user 123.
5. The server validates the incoming data against its schema.
6. The server runs a SQL query (or equivalent) against the database.
7. The database confirms the write.
8. The server sends back a JSON response with the updated user (or an error).

9. React receives the response, updates its cache (via TanStack Query), and re-renders the UI.

Every layer can fail. Every layer needs error handling. The frontend's job is the smallest piece of that chain — but the only piece the user actually sees.

## 10.3 Common backend stacks

The backend can be written in any language. The popular choices, and what you'll recognize them by:

| Stack | Language | Notes |
|---|---|---|
| Node.js + Express | JavaScript/TypeScript | Most common JS backend. Minimal, mature, huge ecosystem. |
| Node.js + Fastify | JavaScript/TypeScript | Faster, more modern alternative to Express. |
| Node.js + NestJS | TypeScript | Opinionated, structured, similar feel to Angular. |
| Next.js API routes | TypeScript | Backend code lives in the same project as the frontend. |
| Python + FastAPI | Python | Fast, modern, type-hinted. Popular for AI/ML-heavy apps. |
| Python + Django | Python | Batteries-included, mature ORM, admin UI built in. |
| Python + Flask | Python | Minimal, flexible. Older default. |
| Go | Go | Fast, simple, great for high-throughput services. |
| Ruby on Rails | Ruby | Convention over configuration. Older but still loved. |
| Java/Kotlin + Spring Boot | Java/Kotlin | Enterprise standard. Verbose, robust. |
| .NET + ASP.NET | C# | Microsoft ecosystem. Strong typing, performance. |
| PHP + Laravel | PHP | Excellent DX for content/CRUD apps. |

**Same job, different syntax**

From the React frontend's perspective, all of these look identical — they expose HTTP endpoints that return JSON. The language behind them only matters when you (or an AI agent) are writing the backend. You can pair a React + TS frontend with a Python or Go backend without any awkwardness.

## 10.4 Databases

Where your data persistently lives. The first division is SQL vs NoSQL:

| Type | Examples | Best for |
|---|---|---|
| Relational (SQL) | PostgreSQL, MySQL, SQLite | Default for most apps. Strong consistency, schemas, joins, transactions. |
| Document (NoSQL) | MongoDB, Firestore, DynamoDB | Flexible schemas, fast writes. Trade-offs in querying and consistency. |
| Key-value | Redis, Memcached | Caching, sessions, rate-limiting, queues. Fast in-memory access. |
| Search | Elasticsearch, Meilisearch, Algolia | Full-text search, faceted filtering. |
| Vector | Pinecone, pgvector, Weaviate, Qdrant | AI embeddings — semantic search, RAG systems. |
| Time-series | InfluxDB, TimescaleDB | Metrics, IoT, financial data. |
| Graph | Neo4j, ArangoDB | Social networks, recommendation engines. |

✓ **Practical tip**

Default to PostgreSQL unless you have a specific reason not to. It's mature, free, supports JSON columns (covering most NoSQL needs), has a vector extension (pgvector), and is supported by every cloud. Most projects that started with MongoDB "for flexibility" wish they'd started with Postgres.

## 10.5 ORMs and data access

An ORM (Object-Relational Mapper) lets you query the database using your programming language's syntax instead of raw SQL. It also generates TypeScript types from your database schema — so the data shape is type-safe end to end.

| Tool | Style | Notes |
|---|---|---|
| Prisma | TypeScript, schema file → client | Most popular TS ORM. Excellent DX, strong types. |
| Drizzle | TypeScript, define schema in code | Lighter, closer to SQL, fast-growing. |
| Kysely | TypeScript query builder, no migrations | Type-safe SQL feel. |
| TypeORM / Sequelize | Older Node ORMs | Still used; less loved than Prisma/Drizzle. |
| SQLAlchemy | Python | The standard Python ORM. |
| Mongoose | Node + MongoDB | Schema and validation for Mongo. |

> ⚠ **Anti-pattern**
> Building SQL queries by string-concatenating user input. This is a SQL injection vulnerability — the textbook security hole. Always use parameterized queries (every ORM does this automatically) or your ORM's query builder.

## 10.6 Authentication and authorization

Two related but different concerns:

- **Authentication** — who is this user? (login, sign-up, sessions)
- **Authorization** — what is this user allowed to do? (roles, permissions, ownership)

Common building blocks:

| Concept | What it is | Where you'll see it |
|---|---|---|
| Session cookie | Server-issued cookie tying request to a session in DB | Traditional web apps; most secure default |
| JWT | Signed token containing user claims, sent on every request | Stateless APIs, mobile clients |
| OAuth / OpenID | Standard for delegating auth ("Sign in with Google") | Social login |
| SAML | Enterprise SSO standard | B2B and corporate apps |
| MFA / 2FA | Multi-factor — password + something else | Security upgrade for sensitive apps |
| RBAC | Role-based access control — roles → permissions | Most apps need some form |
| ABAC | Attribute-based — rules from many attributes | Complex, regulated systems |

Auth providers — services that handle this complexity for you:

| Provider | Style | Notes |
|---|---|---|
| Clerk | Hosted, drop-in React components | Great DX, fastest to integrate |
| Auth0 | Hosted, enterprise-grade | Mature, widely used in B2B |
| Supabase Auth | Part of Supabase BaaS | Free tier; tied to Supabase ecosystem |
| Firebase Auth | Part of Firebase BaaS | Google's offering; tied to Firebase |
| NextAuth / Auth.js | Library, self-hosted | Free, you run it in your Next.js app |
| AWS Cognito | AWS-native | If you're in AWS already |

**✓ Practical tip**

Don't build authentication from scratch. Even "just login with email and password" involves password hashing, session management, rate limiting, password reset flows, email verification, account recovery, and dozens of edge cases. Use Clerk, Auth0, Supabase Auth, or NextAuth. You'll save months and avoid serious security mistakes.

## 10.7 Backend-as-a-Service (BaaS)

A BaaS bundles database, auth, storage, and sometimes serverless functions into one product so you don't have to wire it all together yourself. Great for prototypes, MVPs, side projects, and even production apps that don't need bespoke infrastructure.

| BaaS | Notes | Watch for |
|---|---|---|
| Supabase | Open-source. Postgres + Auth + Storage + Edge Functions + Realtime | Closest to a "real" stack; can self-host later |
| Firebase | Google. Firestore + Auth + Storage + Functions | Vendor lock-in; data model is non-SQL |
| Appwrite | Open-source. Self-hostable | Smaller community than Supabase/Firebase |
| PocketBase | Single binary, SQLite-backed | Brilliant for small projects, side projects |
| Convex | TypeScript-native, reactive backend | Newer, opinionated, great DX for TS-heavy teams |

**When to choose a BaaS**

If your app is mostly CRUD with auth, files, and standard logic — a BaaS will get you to production weeks faster. If your domain has heavy custom logic, third-party integrations, or strict compliance requirements, a custom backend gives you more control.

## 10.8 Cloud and hosting

Where does your app actually run?

### Frontend hosting (static sites and SSR)

| Host | Best for | Notes |
|---|---|---|
| Vercel | Next.js, React SPAs | Made by Next.js creators; best DX for Next |
| Netlify | Static sites, JAMstack | Generous free tier, simple |
| Cloudflare Pages | Static + edge functions | Cheap, fast, global CDN |
| AWS Amplify / S3 + CloudFront | AWS-native static hosting | More control, more setup |
| GitHub Pages | Pure static, simple projects | Free; limited features |

### Backend hosting

| Host | Style | Notes |
|---|---|---|
| Render / Railway / Fly.io | Push code, get a server | Modern Heroku replacements; easiest |
| Vercel / Netlify Functions | Serverless functions | Auto-scaling, pay per call |
| AWS Lambda + API Gateway | Serverless on AWS | Maximum flexibility, steeper learning curve |
| AWS EC2 / GCP Compute / Azure VMs | Full virtual machines | Most control, most ops work |
| Kubernetes | Container orchestration | Heavy; only when you need it |

## 10.9 Other services in a real project

A production app usually pulls in many other services. You'll meet them via API and SDK names:

| Need | Common services |
|---|---|
| Payments | Stripe, Paddle, LemonSqueezy, Razorpay |
| Email (transactional) | Resend, Postmark, SendGrid, AWS SES |
| Email (marketing) | Mailchimp, ConvertKit, Customer.io |
| File storage | AWS S3, Cloudflare R2, Supabase Storage, UploadThing |
| Image transformations | Cloudinary, imgix, Cloudflare Images |

| Need | Common services |
|---|---|
| Analytics | PostHog, Mixpanel, Amplitude, Plausible, GA4 |
| Error tracking | Sentry, Bugsnag, Rollbar |
| Session replay | LogRocket, FullStory, PostHog |
| Feature flags | LaunchDarkly, Statsig, PostHog, ConfigCat |
| Search | Algolia, Meilisearch, Typesense |
| Real-time | Pusher, Ably, Supabase Realtime, Liveblocks |
| Background jobs | Inngest, Trigger.dev, BullMQ, Temporal |
| AI / LLM | Anthropic, OpenAI, Vercel AI SDK, LangChain |

## 10.10 A realistic AI-era stack for a new project

If you're starting a new SaaS app today with AI agents doing most of the typing, a common modern combination is:

- **Framework:** Next.js (App Router) — React frontend + API routes in one project.
- **Language:** TypeScript everywhere, strict mode.
- **Styling:** Tailwind CSS + shadcn/ui — utility classes plus copy-paste accessible components.
- **Database:** PostgreSQL hosted on Supabase, Neon, or Railway.
- **ORM:** Prisma or Drizzle.
- **Auth:** Clerk or NextAuth (or Supabase Auth if you're using Supabase).
- **Data fetching on the client:** TanStack Query (or framework loaders).
- **Validation:** Zod everywhere — API input, forms, env vars.
- **Forms:** React Hook Form + Zod.
- **Payments:** Stripe.
- **Email:** Resend.
- **Hosting:** Vercel (frontend + serverless functions). Database on its own provider.
- **Error tracking:** Sentry.
- **Analytics:** PostHog or Plausible.

This is a strong default. It's well-trodden, has great documentation, and AI agents have seen thousands of examples — so the help they offer is dependable. Diverge from it when a real project requirement asks you to.

## 10.11 Monorepos: one repo, many packages

Larger projects often split code across multiple packages — web app, admin app, shared UI library, backend, mobile app — all in one Git repository. This is a monorepo. The tools:

- Turborepo — fast incremental builds across packages, simple config.
- Nx — more powerful, more complex, popular at large companies.
- pnpm workspaces / npm workspaces / yarn workspaces — the underlying package-linking layer.

Benefits: share TypeScript types between frontend and backend, refactor across the boundary, single CI pipeline. Cost: more upfront setup; AI agents can occasionally get confused about which workspace they're in.

## 10.12 End-to-end type safety

The holy grail of full-stack TypeScript: types flow from the database, through the backend, all the way to React components. Refactor a column in the DB, the React UI fails to compile. Several approaches:

| Approach | How it works | Best when |
|---|---|---|
| tRPC | Backend functions called directly from the frontend with full type inference | Full-stack TypeScript, one team |
| GraphQL Codegen | Generate TS types and hooks from the GraphQL schema | Using GraphQL, possibly polyglot stack |
| OpenAPI Codegen | Generate types and clients from OpenAPI spec | REST APIs, polyglot stack |
| Shared types in a monorepo | Backend and frontend import the same types from a shared package | Monorepo, custom protocol |
| Zod + tRPC / Hono | Validation schemas double as type definitions | Modern TS-first stack |

> ✓ **Practical tip**
> End-to-end type safety is one of the most under-rated upgrades available to AI-assisted coding. When the agent makes a change in the backend, the frontend's compile errors tell you exactly which files need to update. The feedback loop is fast, precise, and largely automatic.

## 10.13 CI/CD: how code gets to production

Continuous Integration / Continuous Deployment automates the path from "I committed code" to "it's running in production." A typical pipeline:

1. Developer pushes a branch to GitHub.

2. GitHub Actions (or GitLab CI, CircleCI) runs: install deps, type-check, lint, run tests, build.
3. If all green, a preview deployment goes up (Vercel and Netlify do this automatically per branch).
4. After review and merge, the main branch deploys to production.
5. Monitoring tools (Sentry, Datadog) watch for errors and alert if something spikes.

**✓ Practical tip**

Set up the CI pipeline before you have anything to ship. The earlier those checks exist, the earlier they catch problems. AI agents introduce small mistakes constantly; CI is your filter.

# Part 11 — Working with AI Coding Agents

Everything above gives you the vocabulary. This part is about the workflow — how to be effective when an AI is doing the typing.

## 11.1 The three modes of AI coding

| Mode | What it looks like |
|---|---|
| Autocomplete | AI suggests the next few lines as you type (Copilot-style) |
| Chat-driven | You describe what you want; AI proposes a diff; you accept or refine |
| Agentic | AI plans and executes multi-step tasks across files (Claude Code, Cursor agents) |

As autonomy increases, your responsibility shifts from "correct the suggestion" to "verify the result." The mental shift is the hardest part of vibe engineering.

## 11.2 Setting up the agent for success

AI agents work much better with context. A few investments pay off enormously:

- **A CLAUDE.md (or AGENTS.md) at the repo root.** Describe the project's purpose, tech stack, conventions, where things live, and any rules — "always write tests for new functions," "use Zod for schema validation," "prefer named exports."
- **Strict TypeScript.** Strict mode catches whole classes of AI errors at compile time.
- **Good lint rules.** react-hooks/exhaustive-deps, no-explicit-any, no-floating-promises — these turn into automatic review by the IDE.
- **A test runner that's fast and easy to invoke.** Agents that can run tests and read the output catch their own mistakes.
- **Pre-commit hooks.** Husky + lint-staged runs the linter, formatter, and type check before code can be committed.
- **A working development environment.** If the AI can run the dev server, see the page, and read logs, it can fix its own mistakes. If it can't, you become the eyes — slower and more expensive.

## 11.3 How to write good prompts

Bad prompt: "Add a settings page."

Good prompt: "Add a /settings route in src/features/settings/. It should let the user update their display name and email. Use React Hook Form with Zod validation; the schema lives in settings.schema.ts. On submit, call updateUser from src/api/user.ts — that mutation is already

set up with TanStack Query. Show a toast on success using our existing useToast hook. Match the visual style of the existing /profile page."

Specifics that matter:

- Where the code goes (folder, file).
- Which existing utilities to use.
- The user-facing behavior — happy path, error path, edge cases.
- The shape of the data flowing in and out.
- Visual or interaction precedents to follow.

> ✓ **Practical tip**
> Before a non-trivial task, ask the agent to write a plan first — what files it will create or modify, in what order, and why. Read and approve the plan before any code is written. This catches 80% of misunderstandings up front, where they're cheap.

## 11.4 Reviewing AI-generated code

A practical checklist when an agent shows you a diff:

1. Does it actually solve the problem I described? (Surprisingly often, no — the agent solved an adjacent problem.)
2. Does it use any new dependencies? Are they necessary? Are they maintained?
3. Are the types specific, or did it reach for any?
4. Are loading, error, and empty states handled?
5. Is async work cancelled on unmount where appropriate?
6. Are useEffect dependencies complete?
7. Is state updated immutably?
8. Are there tests? Do they test behavior, not implementation?
9. Are there security issues — leaked secrets, dangerouslySetInnerHTML, untrusted data in URLs?
10. Is there dead code, console.log statements, or TODOs that were left behind?
11. Did the agent change anything outside the scope you asked for?

## 11.5 Common AI failure modes

| Failure mode | What it looks like |
|---|---|
| Hallucinated API | Calls a function or method that doesn't exist in the library |
| Outdated patterns | Uses class components, deprecated lifecycle methods, or old hook patterns |

| Failure mode | What it looks like |
|---|---|
| Over-abstraction | Wraps simple code in three layers of generics and configuration |
| Silent error swallowing | try/catch that swallows the error without logging or handling |
| State that doesn't update | Mutation, missing dependencies, or wrong useState pattern |
| Missing accessibility | No alt text, no labels, no keyboard support, wrong heading order |
| Plausible but wrong types | TypeScript that compiles but doesn't match the real API response |
| Phantom files | Imports something it never actually created |
| Scope creep | Refactors unrelated code while doing a simple task |
| Sycophancy | Agrees too readily that its broken code is correct after a vague nudge |

## 11.6 When to push back

Agents are accommodating. They will rewrite the same flawed approach five different ways if you ask. That's bad for you. Push back when:

- The solution feels more complex than the problem warranted.
- It introduces a new library to do something the existing stack already does.
- You don't understand a part of the diff. Ask the agent to explain that part before accepting.
- Tests are missing or test only the happy path.
- It's making changes outside the scope you described.
- It tells you something is impossible. Sometimes true; often it's just hard. Ask for the trade-offs.

**The single best habit**
Before accepting any non-trivial change: ask the agent to summarize what it did, why, and what could go wrong. Read that summary against the diff. Mismatches between explanation and code are the most reliable signal that something is off.

## 11.7 The verification loop

After every meaningful AI-generated change, the loop is:

1. Type-check passes (tsc or your IDE)

2. Lint passes
3. Tests pass — including new ones for the new behavior
4. App still runs in the dev server
5. Happy path works in the browser
6. At least one error path works
7. If the change touches auth or money, manually verify both as the intended user and as a malicious user

Skipping any of these saves seconds and costs hours later. The agent can run all of these for you; you just have to insist.

## 11.8 Working in small commits

Small, focused commits make AI work safer. Each commit:

- Does one thing and has a descriptive message.
- Passes all checks on its own.
- Can be reverted cleanly if it turns out to be wrong.

> ✓ **Practical tip**
> After a successful AI change, commit immediately. If the next change breaks something, you can git reset and try a different prompt rather than untangling a mess. Cheap rollback is freedom to experiment.

## 11.9 What to keep in your head versus look up

You do not need to memorize syntax. You need to recognize patterns and concepts. Specifically:

- Always carry: the mental model of components, props, state, hooks, the data flow direction.
- Always carry: the categories of state and which tool fits each.
- Always carry: what a good test looks like vs a brittle one.
- Always carry: the security and accessibility non-negotiables.
- Look up: exact API surfaces, library options, configuration syntax — the agent or docs will be faster than your memory.

## 11.10 The most important habit

After the AI ships something, build a small test by hand. Click through it. Try the wrong inputs. Try the happy path. Try refreshing in the middle. Try with the network throttled. The AI can write tests; only you can run the app as a user.

The thirty seconds you spend acting like a user is the single highest-leverage practice in AI-assisted development. It catches the things that compile, lint, and pass — and still don't work.

# Glossary

Quick reference for terms you'll encounter when working with React, TypeScript, backends, and AI coding agents.

| | |
|---|---|
| **Accessibility (a11y)** | Designing apps so they work for people with disabilities — screen readers, keyboard-only navigation, color blindness, low vision. Often abbreviated a11y (eleven letters between a and y). |
| **Agent (AI)** | An AI system that can plan and execute multi-step tasks autonomously, often calling tools (file edits, shell commands, web searches) along the way. |
| **API** | Application Programming Interface. The contract by which one piece of software talks to another — usually over HTTP for web apps. |
| **Async / await** | JavaScript syntax for working with Promises in a linear, readable style. await pauses the function until the Promise resolves. |
| **Auth provider** | A service that handles authentication for you — Clerk, Auth0, Supabase Auth, Firebase Auth. |
| **Authorization** | Deciding what a logged-in user is allowed to do. Distinct from authentication (who they are). |
| **Babel** | A JavaScript transpiler. Converts modern JS and JSX into older JS that all browsers understand. Now often replaced by SWC or esbuild. |
| **BaaS** | Backend-as-a-Service. Bundles database, auth, and storage so you don't run your own backend. Supabase, Firebase, Appwrite. |
| **Bundler** | A tool that combines many source files into a few optimized output files. Vite, Webpack, esbuild. |
| **Callback** | A function passed as an argument to another function, to be called later. |
| **CDN** | Content Delivery Network. A globally distributed cache that serves static files (CSS, JS, images) from a location near the user. |
| **Children prop** | A built-in prop that holds whatever JSX is nested inside a component's opening and closing tags. |
| **CI/CD** | Continuous Integration / Continuous Deployment. Automated systems that run tests and ship code on every commit. |
| **Class component** | The old way to write React components, using ES6 classes. Replaced by function components with hooks. |
| **Client component** | In Next.js with React Server Components: a component that runs in the browser, marked with "use client". |
| **CLI** | Command Line Interface. The shell tools you invoke by typing, e.g. npm, git, the build tool. |
| **Compound component** | A pattern where a parent and matched children work together — used by libraries like Radix and shadcn. |
| **Component** | A reusable piece of UI in React. A function that returns JSX. |

| | |
|---|---|
| **Context** | React's mechanism for sharing data across the component tree without prop drilling. |
| **Controlled component** | A form input whose value is driven by React state. |
| **Cookie** | A small piece of data the browser stores per site and sends with each request. Used for sessions. |
| **CORS** | Cross-Origin Resource Sharing. A browser security mechanism. If you see CORS errors, the backend isn't allowing requests from your frontend's domain. |
| **Core Web Vitals** | Google's user-experience metrics: LCP, INP, CLS. |
| **CRUD** | Create, Read, Update, Delete. The four basic data operations most apps perform. |
| **CSR** | Client-Side Rendering. JavaScript renders the page in the browser after download. |
| **CSS modules / CSS-in-JS / Tailwind** | Three competing styling approaches. Tailwind (utility classes in HTML) is currently dominant. |
| **Dependency array** | The second argument to useEffect, useMemo, useCallback. The hook re-runs when any value in the array changes. |
| **Destructuring** | Syntax for pulling values out of objects or arrays into variables. |
| **Discriminated union** | A union type where each variant has a shared field telling TypeScript which variant you're in. |
| **DNS** | Domain Name System. Maps domain names to IP addresses. |
| **DOM** | Document Object Model. The browser's in-memory tree of the page. React's job is to manage this tree. |
| **Effect** | React's term for a side effect — anything outside the pure render flow. Managed by useEffect. |
| **End-to-end type safety** | Types flow unbroken from the database, through the backend, into the React frontend. Achieved with tRPC, GraphQL Codegen, OpenAPI Codegen, or shared types. |
| **ESLint** | The standard JavaScript linter — flags bugs and style violations. |
| **E2E test** | End-to-end test. Drives a real browser through user flows. Playwright, Cypress. |
| **Fetch** | The browser's built-in API for making HTTP requests. |
| **Fragment** | An empty JSX tag <>…</> that lets a component return multiple elements without a wrapper div. |
| **Framework** | A more opinionated layer on top of React: Next.js, Remix, TanStack Start. Includes routing, data loading, and often SSR. |
| **Function component** | The modern way to write React components — a plain function returning JSX. |
| **Generic (TypeScript)** | A type parameter, e.g. T in Array<T>. Lets code work with multiple types while keeping type safety. |

| | |
|---|---|
| **GraphQL** | An alternative to REST. Clients request the exact shape of data they want from a single endpoint. |
| **Hook** | A special function in React that lets function components use state and other features. Always starts with use. |
| **HMR** | Hot Module Replacement. The dev server swaps changed code into the running app without a full reload. |
| **HOC** | Higher-Order Component. A function that takes a component and returns a new component. An older pattern; custom hooks have largely replaced it. |
| **IDE** | Integrated Development Environment. VS Code, Cursor, JetBrains. Where you write code. |
| **Immutability** | Treating values as unchanging — instead of modifying an object, create a new one. Required for React state. |
| **Interface (TypeScript)** | A way to describe the shape of an object. Similar to type, slightly different mechanics. |
| **JSON** | JavaScript Object Notation. The standard data format for APIs. |
| **JSX** | JavaScript XML. The HTML-like syntax inside React components. Compiled to function calls. |
| **JWT** | JSON Web Token. A signed, base64-encoded string carrying user claims. Used for stateless auth. |
| **Key prop** | A unique identifier React uses to track items in a list across renders. |
| **Lazy loading** | Loading code only when it's needed, instead of upfront. React.lazy + Suspense. |
| **Linter** | A static analysis tool that flags issues in code without running it. |
| **Memoization** | Caching a computed value so it doesn't get recalculated unnecessarily. React.memo, useMemo, useCallback. |
| **Middleware** | Code that intercepts and processes requests or responses — common in Redux and server frameworks. |
| **Migration** | A script that evolves the database schema. Tools: Prisma Migrate, Drizzle Kit, Alembic, etc. |
| **Module** | A file with exports and imports. The unit of code organization in modern JavaScript. |
| **Monorepo** | One Git repository containing multiple packages — web, backend, shared, etc. Tools: Turborepo, Nx. |
| **MSW** | Mock Service Worker. Intercepts network requests in tests so you can fake API responses. |
| **Mutation** | (1) Changing data in place — usually wrong in React. (2) An API call that changes server state — the opposite of a query. |
| **Next.js** | The most popular React framework. File-system routing, SSR, RSC, and more. |

| | |
|---|---|
| **Node.js** | JavaScript runtime outside the browser. Needed for React development tooling. |
| **NoSQL** | Non-relational databases. MongoDB, DynamoDB, Firestore. |
| **npm** | Node Package Manager. The default tool for installing JavaScript dependencies. |
| **OAuth** | Open standard for delegated authorization. "Sign in with Google" uses OAuth. |
| **Optimistic update** | Updating the UI immediately as if a server call succeeded, then rolling back if it didn't. |
| **ORM** | Object-Relational Mapper. Lets you query a database using your language's syntax instead of raw SQL. Prisma, Drizzle, SQLAlchemy. |
| **package.json** | The manifest file describing a project's dependencies and scripts. |
| **Pagination** | Splitting a long list of results into pages. Offset/limit or cursor-based. |
| **Playwright** | Modern end-to-end testing tool from Microsoft. |
| **pnpm / yarn / bun** | Alternative package managers to npm. Same purpose. |
| **Polyfill** | Code that adds modern JavaScript features to older environments that lack them. |
| **PostgreSQL (Postgres)** | The most popular open-source relational database. Strong default for most projects. |
| **Prettier** | An opinionated code formatter. Ends style debates by formatting everything the same way. |
| **Prisma** | The leading TypeScript ORM. Schema file generates a fully-typed client. |
| **Promise** | A JavaScript object representing a future value — the result of an async operation. |
| **Prop drilling** | Passing props through many intermediate components just to reach a deep child. |
| **Props** | Arguments passed to a component by its parent. Read-only inside the component. |
| **Pure function** | A function with no side effects that returns the same output for the same input. React components should be pure. |
| **RAG** | Retrieval-Augmented Generation. Fetching relevant context, often from a vector DB, before asking an LLM. |
| **RBAC** | Role-Based Access Control. Authorization model using user roles. |
| **React DevTools** | A browser extension that lets you inspect the React component tree, props, and state. |
| **Re-render** | When React calls a component function again to produce updated JSX, in response to a state or prop change. |
| **Reducer** | A function that takes the current state and an action, and returns the next state. The core of useReducer and Redux. |

| | |
|---|---|
| **Ref** | A mutable container that survives across renders, often used to access DOM elements. useRef. |
| **Remix** | A React framework focused on web standards. Now closely aligned with React Router. |
| **REST** | Representational State Transfer. The traditional style for HTTP APIs. |
| **RSC** | React Server Components. Components that render only on the server, never shipping their code to the browser. |
| **Schema validation** | Runtime checking that data matches an expected shape. Zod is the leading library. |
| **Serverless** | Code runs in short-lived functions on demand — you don't manage servers. AWS Lambda, Vercel Functions. |
| **Session** | An authenticated period for a user. Often tracked via a session cookie or token. |
| **shadcn/ui** | A collection of accessible React components you copy into your project. Not an npm dependency; you own the code. |
| **SPA** | Single-Page Application. Loads once, then JavaScript handles navigation. |
| **SQL injection** | A vulnerability from inserting user input directly into SQL. Always use parameterized queries. |
| **SSG** | Static Site Generation. Pages are built at build time and served as static files. |
| **SSO** | Single Sign-On. One login covers multiple apps. Often via SAML or OAuth. |
| **SSR** | Server-Side Rendering. The server renders the HTML for each request. |
| **State** | Data that changes over time inside a component or app. Drives the UI. |
| **Storybook** | A tool for developing and showcasing UI components in isolation. |
| **Stripe** | The dominant payments API for web apps. |
| **Supabase** | Open-source BaaS — Postgres, auth, storage, realtime. |
| **Suspense** | A React feature for declaratively handling async work — usually paired with lazy loading or data libraries. |
| **SWC** | A fast Rust-based transpiler. Used by Next.js and others as a replacement for Babel. |
| **Tailwind CSS** | A utility-first CSS framework. You compose styles by adding small class names directly in JSX. |
| **TanStack Query** | The dominant server-state library. Formerly known as React Query. |
| **tRPC** | End-to-end type safety for TypeScript backends and frontends — no separate API schema. |
| **Tree shaking** | A bundler optimization that removes code that's imported but never used. |

| | |
|---|---|
| **Transpile** | Convert source code from one form to another — typically TypeScript or modern JS into older browser-compatible JS. |
| **TSC** | The TypeScript compiler. Checks types and emits JavaScript. |
| **tsconfig.json** | The TypeScript project configuration file. Controls strictness, module system, output target. |
| **Type assertion** | Telling TypeScript "trust me, this value is of type X." Uses the as keyword. Use sparingly. |
| **Type inference** | TypeScript's ability to deduce types from context without explicit annotations. |
| **Union type** | A type that allows one of several options: string \| number. |
| **Unit test** | A test of a single function or component in isolation. |
| **Uncontrolled component** | A form input that manages its own value internally — React reads it via a ref. Less common. |
| **URL state** | App state that lives in the URL — current route, query parameters. |
| **Utility type** | Built-in TypeScript helper that transforms a type. Partial, Pick, Omit, Record, ReturnType. |
| **Vector database** | Stores high-dimensional vectors (AI embeddings) and finds nearest neighbors. Pinecone, pgvector. |
| **Vite** | The default modern build tool for React projects. Fast dev server, simple config. |
| **Vitest** | A test runner built for Vite. API compatible with Jest. |
| **Webhook** | An HTTP callback. The other service POSTs to your URL when something happens. |
| **WebSocket** | A persistent two-way connection between browser and server. Used for real-time features. |
| **Webpack** | The legacy bundler. Still widely used but being displaced by Vite and others. |
| **XSS** | Cross-Site Scripting. A vulnerability from injecting user-controlled content as HTML. React escapes by default; dangerouslySetInnerHTML opens the door. |
| **Yarn** | An alternative npm. Same job, different tool. |
| **Zod** | A schema-validation library that doubles as a TypeScript type generator. Now near-default. |
| **Zustand** | A lightweight global state library. Increasingly popular as a simpler alternative to Redux. |

# Appendix — Command Cheat Sheet

The shell commands you'll see most often when working with AI agents on a React + TypeScript project.

## Project setup

```
# Create a new React + TS project with Vite
npm create vite@latest my-app -- --template react-ts
cd my-app
npm install
npm run dev

# Or with Next.js
npx create-next-app@latest my-app --typescript --tailwind --eslint
```

## Daily workflow

```
npm run dev          # start dev server
npm run build        # build production bundle into dist/
npm run preview      # preview the production build locally
npm run lint         # run ESLint
npm run test         # run tests
npm run typecheck    # run TypeScript compiler in check-only mode (if configured)

npm install <pkg>      # add a runtime dependency
npm install -D <pkg>   # add a dev-only dependency
npm uninstall <pkg>    # remove a dependency
npm update             # update dependencies within version ranges
npm outdated           # list dependencies behind their latest version
```

## Git essentials

```
git status
git diff
git add .
git commit -m "message"
git push
git pull
git checkout -b feature/new-thing
git log --oneline -20
git reset --hard HEAD     # discard all uncommitted changes — careful!
git revert <commit>       # safely undo a committed change
```

## Useful one-liners

```
# Show what version of Node and npm you have
node --version
npm --version

# Check what's using port 5173 (when dev server says 'port in use')
lsof -i :5173        # macOS/Linux
```

```
# Nuke node_modules and reinstall when something is weird
rm -rf node_modules package-lock.json && npm install
```

# Closing Thought

Fluency in React, TypeScript, and the full stack around them isn't about writing every line from memory. It's about having a strong enough mental model that, when you see code — yours, a colleague's, or an AI's — you can quickly tell whether it's right, where it can break, and what's missing.

Read this document once. Build something small with an AI agent — a personal task tracker, a small dashboard, a tool you'd actually use. Come back to the relevant section when a term confuses you. Within a handful of projects, the vocabulary becomes yours, and the agent becomes a power tool instead of a black box.

The barrier to building software has collapsed. What hasn't collapsed is the value of understanding what you're shipping. This document is a deposit on that — the rest is reps.